

Biscuit Optimization Project: Choices

Introduction

This document provides an in-depth explanation of the strategies we implemented to solve the biscuit placement optimization problem. Our goal was to maximize the value of biscuits placed on a dough roll while adhering to defect constraints and minimizing penalties for unused spaces. Throughout the process, we explored various approaches, making deliberate choices and analyzing their outcomes to understand the strengths and weaknesses of each method.

Problem Representation

Defining the Dough Roll

We represented the dough roll as a one-dimensional array of length 500, where each index corresponds to a unit of space. This representation allowed us to efficiently track occupied positions and validate placements against constraints.

Defects Representation

Defects were stored in a dictionary where each key represented a position on the roll, and the associated values contained defect counts for classes 'a,' 'b,' and 'c.' This structure enabled us to quickly access defect data during placement validation, ensuring that we didn't exceed the thresholds for any biscuit.

Biscuit Representation

Each biscuit was represented as a dictionary with attributes such as `length`, `value`, and `thresholds` for defect classes. This design allowed us to dynamically validate constraints for each biscuit type during the placement process.

Strategies and Justifications

Greedy Strategy by Biscuit Size

Concept: The greedy approach sorts biscuits by descending size and attempts to place them sequentially on the roll. We chose this method because it prioritizes larger biscuits, which are harder to fit once smaller biscuits occupy fragmented spaces.

Key Design Decisions:

1. **Sorting by Size:** Larger biscuits were placed first to ensure that large contiguous spaces were used efficiently.
2. **Sequential Placement:** This straightforward strategy ensured that we could quickly fill available spaces with minimal computational overhead.

Why We Chose This Approach: The simplicity of the greedy algorithm made it an excellent baseline for comparison with more complex strategies.

Strengths and Limitations:

- **Strength:** Very efficient computationally, as it involves sorting and a linear scan for placement.
- **Limitation:** Left gaps on the roll that could not be filled by smaller biscuits, leading to suboptimal results.

Genetic Algorithm (GA)

Concept: The GA uses evolutionary principles to optimize biscuit placements. We iteratively improved a population of solutions using selection, crossover, mutation, and elitism.

Key Design Decisions:

1. **Selection (Tournament):** This allowed us to focus on high-quality solutions while maintaining diversity in the population.
2. **Crossover (Single-Point):** Mixed promising features from parent solutions, generating diverse offspring.
3. **Mutation:** Introduced random changes to help the algorithm escape local optima.
4. **Elitism:** Preserved top-performing solutions across generations to ensure steady improvement.

Why We Chose This Approach: The GA's iterative nature made it well-suited for exploring complex solution spaces.

Strengths and Limitations:

- **Strength:** Balanced exploration and refinement, making it robust and adaptable.
- **Limitation:** Required careful parameter tuning to avoid stagnation or excessive randomness.

Dynamic Programming (DP)

Concept: DP guarantees the optimal solution by solving the problem recursively and storing intermediate results to avoid redundant calculations. This approach allowed us to exhaustively explore all valid placements.

Key Design Decisions:

1. **State Definition:** Each state was defined by the starting position on the roll and the remaining space, simplifying subproblem definitions.

2. **Memoization:** Cached results for each state significantly reduced redundant computations.

Why We Chose This Approach: DP was chosen as our benchmark for optimality. By guaranteeing the best possible result, it allowed us to evaluate the performance of heuristic methods.

Strengths and Limitations:

- **Strength:** Guaranteed the best solution by exploring every possible placement.
- **Limitation:** Computationally intensive and required significant memory.

Simulated Annealing (SA)

Concept: SA explores the solution space probabilistically, accepting worse solutions with a probability that decreases over time. **This helps the algorithm escape local optima and converge to a near-optimal solution.**

Key Design Decisions:

1. **Cooling Schedule:** Designed to balance exploration and exploitation by gradually reducing randomness.
2. **Neighbor Generation:** Focused on small, feasible adjustments to placements to explore new solutions while maintaining constraints.

Why We Chose This Approach: SA's ability to escape local optima made it a valuable complement to deterministic methods.

Strengths and Limitations:

- **Strength:** Effective at avoiding local optima and exploring diverse solutions.
- **Limitation:** Performance was highly sensitive to parameter tuning.

Conclusion

Each approach reflected deliberate choices and trade-offs tailored to specific goals within the optimization problem. **Dynamic Programming achieved the best results and validated the effectiveness of heuristic strategies, while the Genetic Algorithm provided a practical balance between computational effort and solution quality.** Simulated Annealing added depth to our exploration, and the Greedy Strategy served as a useful baseline.

Through this project, we learned the importance of combining diverse methodologies to address complex optimization challenges. Each strategy brought unique strengths to the table, showcasing the value of thoughtful algorithmic design and analysis.